



**POLITECHNIKA
BYDGOSKA**

Wydział Telekomunikacji,
Informatyki i Elektrotechniki

Zaawansowane programowanie obiektowe - laboratorium

Laboratorium nr: 1

**Tytuł: IMPLEMENTACJA WZORCA PRODUCENT -
KONSUMENT GENERUJĄCEGO STATYSTYKI SŁÓW Z
WYKORZYSTANIEM PROGRAMOWANIA
FUNKCYJNEGO I WSPÓLBIEŻNEGO**

Szymon Banaszek, 1 grupa

Nr legitymacji: 124983

Prowadzący dr inż. Damian Szczegielniak

Bydgoszcz, dnia: 10.04.2026

Ćwiczenie

Zaimplementuj aplikację, która będzie analizowała zawartość plików znajdujących się we wskazanym katalogu głównym. W zadaniu należy skorzystać z kolejki blokującej np. `LinkedBlockingQueue` (z pakietu `java.util.concurrent`) o ograniczonej pojemności. Jeden wątek (producent) powinien przechodzić przez drzewo podkatalogów i plików wyszukując pliki tekstowe, a także wstawiać ścieżki na nie wskazujące do kolejki blokującej (ścieżka powinna być obiektem klasy `java.nio.file.Path` opakowanym w `java.util.Optional` np. `Optional optPath = Optional.ofNullable(path);`). Kolejne dwa wątki (konsumenci) będą pobierały/usuwały obiekty z kolejki i wyszukiwały dziesięć najczęściej występujących wyrazów we wskazanych plikach. W tego typu zadaniu prawdopodobnie wątek producenta szybko wypełni kolejkę obiektami wskazującymi konkretne pliki i zostanie zablokowany do czasu, gdy wątki odbierające elementy wykonają swoją pracę i ponownie opróżnią zapełnioną kolejkę.

Struktura projektu

- [Main.java](#) – uruchamia aplikację i GUI.
- [MainFrame.java](#) – logika GUI oraz obsługa wątków producenta i konsumentów.
- [WordStatsCalculator.java](#) – analiza i zliczanie słów w plikach.

Najważniejsze fragmenty kodu

1. Uruchomienie GUI

```
1. public class Main {
2.     public static void main(String[] args) {
3.         SwingUtilities.invokeLater(() -> {
4.             MainFrame window = new MainFrame();
5.             window.frame.setVisible(true);
6.         });
7.     }
8. }
```

Kod ten uruchamia okno aplikacji w wątku Swing

2. Analiza słów w pliku (strumienie)

```
1. public static Map<String, Long> getLinkedCountedWords(Path path, int wordsLimit) {
2.     try (BufferedReader reader = Files.newBufferedReader(path)) {
3.         return reader.lines()
4.             .flatMap(line -> Arrays.stream(line.split("\\s+")))
5.             .map(String::toLowerCase)
6.             .map(word -> word.replaceAll("[^a-z0-9ąćęóźśńł]", ""))
7.             .filter(word -> word.matches("[a-z0-9ąćęóźśńł]{3,}")
8.             .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
9.             .entrySet().stream()
10.            .sorted(Map.Entry.comparingByValue(Comparator.reverseOrder()))
11.            .limit(wordsLimit)
12.            .collect(Collectors.toMap(
13.                Map.Entry::getKey,
14.                Map.Entry::getValue,
15.                (k, v) -> { throw new IllegalStateException("Duplikat klucza"); },
16.                LinkedHashMap::new
17.            ));
18.    } catch (IOException e) {
19.        throw new RuntimeException(e);
20.    }
21. }
22.
```

Ten fragment odpowiada za:

- Wczytanie pliku,
- Podział na słowa, filtrację, normalizację,
- Grupowanie i zliczanie,
- Sortowanie i ograniczenie do N najczęstszych słów.

3. Współbieżność – producent i konsumenci

```
1. BlockingQueue<Optional<Path>> kolejka = new LinkedBlockingQueue<>(10);
2.
3. // PRODUCENT
4. Runnable producent = () -> {
5.     Files.walkFileTree(Paths.get("files"), new SimpleFileVisitor<Path>() {
6.         public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException
7.         {
8.             if (file.toString().endsWith(".txt")) {
9.                 kolejka.put(Optional.of(file));
10.            }
11.            return FileVisitResult.CONTINUE;
12.        }
13.    });
14.    for (int i = 0; i < liczbaKonsumentow; i++) {
15.        kolejka.put(Optional.empty());
16.    }
```

```

16. };
17.
18. // KONSUMENT
19. Runnable konsument = () -> {
20.     while (!Thread.currentThread().isInterrupted()) {
21.         Optional<Path> optPath = kolejka.take();
22.         if (!optPath.isPresent()) break;
23.         Map<String, Long> stats = WordStatsCalculator.getLinkedCountedWords(optPath.get(),
liczbawYrazowStatystyki);
24.         System.out.println("Plik: " + optPath.get().getFileName() + " -> " + stats);
25.     }
26. };
27.

```

- Producent przeszukuje katalog [files](#) i wrzuca ścieżki plików do kolejki.
- Konsumenci pobierają ścieżki, analizują pliki i wypisują statystyki.
- Po zakończeniu producent wrzuca do kolejki tzw. "poison pill" (`Optional.empty()`), aby zakończyć pracę konsumentów.

4. GUI – obsługa przycisków

```

1. JButton btnStart = new JButton("Start");
2. btnStart.addActionListener(e -> getMultiThreadedStatistics());
3.
4. JButton btnStop = new JButton("Stop");
5. btnStop.addActionListener(e -> {
6.     fajrant.set(true);
7.     producentFuture.forEach(f -> f.cancel(true));
8. });

```

- Przycisk "Start" uruchamia analizę,
- Przycisk "Stop" przerywa pracę wątków

Wnioski.

- Nauczyłem się praktycznie stosować programowanie współbieżne w Javie, zwłaszcza wzorzec producent-konsument.
- Wykorzystałem nowoczesne API Javy (strumienie, kolekcje, Swing), dzięki czemu kod jest czytelny i łatwy do rozbudowy.
- Zadanie pokazuje praktyczne użycie programowania współbieżnego w Javie (wzorzec producent-konsument).